A Trimester – 1 Class Project Report on

SMART TRAVELLING SHORTEST PATH BOOKING PLATFOEM

Submitted in the partial fulfillment of the requirements for

26CS5101E – Computational Thinking and Programming

Master of Technology In

Artificial Intelligence & Data Science

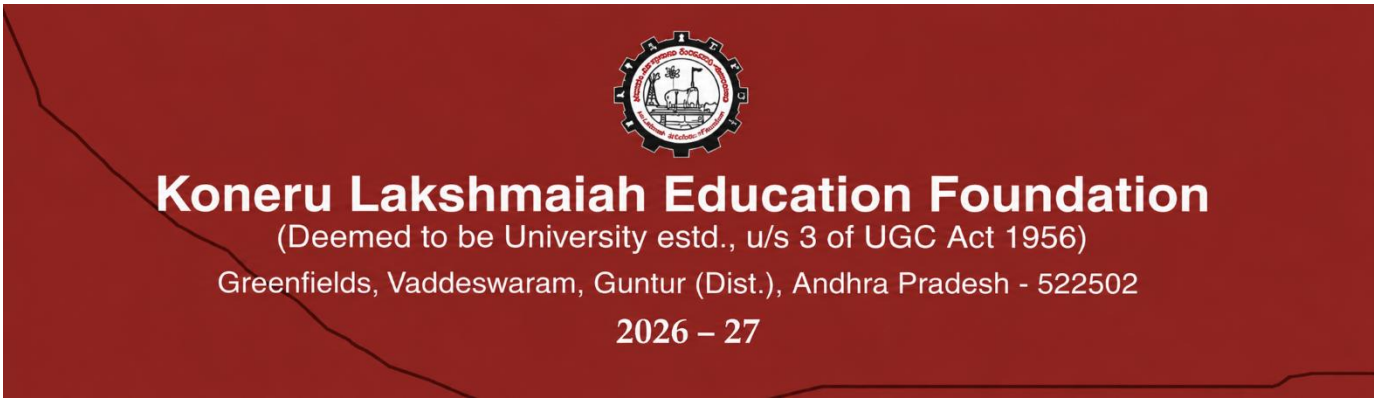
By

Student ID	Student Name
2601150020	SHAIK GOUSE BASHA

under the supervision of

Dr. G. Syam Prasad

Professor



N

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



CERTIFICATE

This is to certify that the Trimester – 1 Class Project Report entitled "**SMART TRAVELLING SHORTEST PATH BOOKING PLATFOEM**" is being submitted by **SHAIK GOUSE BSAHA 2601150020** submitted in partial fulfillment the requirements for the course 26CS5101E – Computational Thinking and Programming, Master of Technology in Artificial Intelligence and Data Science to the K L University is a record of Bonafide work carried out under our guidance and supervision during the academic year 2026 – 2027.

Dr. G. Syam Prasad

Project Supervisor

Signature

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



DECLARATION

The Trimester -1 Class Project Report entitled "**SMART TRAVELLING SHORTEST PATH BOOKING PLATFOEM**" is a record of **SHAIK GOUSE BSAHA - 2601150020** Bonafide work of submitted in partial fulfillment the requirements for the course 26CS5101E – Computational Thinking and Programming, Master of Technology in Artificial Intelligence and Data Science to the K L University.

SHAIK GOUSE BASHA - 2601150020

ACKNOWLEDGEMENT

It is great pleasure for me to express my gratitude to our honorable President **Sri. Koneru Satyanarayana**, for giving the opportunity and platform with facilities in accomplishing the trimester - 1 class project report.

I express sincere gratitude to HOD of AI&DS Dept. **Dr. M. Venkata Naresh** for his leadership and constant motivation provided in successful completion of our academic semester. I record it as my privilege to deeply thank for providing us the efficient faculty and facilities to make our ideas into reality.

I express my sincere thanks to our project supervisor **Dr. G. Syam Prasad** for his novel association of ideas, encouragement, appreciation, and intellectual zeal which motivated us to venture this project successfully.

Finally, it is pleased to acknowledge the indebtedness to all those who devoted themselves directly or indirectly to make this project report success.

ABSTRACT

The Smart Travelling Shortest Path Booking Platform is a Python-based application designed to help users identify suitable travel routes between a source and a destination. The system focuses on finding efficient routes, comparing alternative paths, reducing unnecessary travel distance or time, and supporting a convenient booking workflow.

Traditional travel planning may require users to compare multiple routes, transportation options, journey durations, and booking details manually. The proposed platform simplifies this process by representing locations as nodes and travel connections as edges in a graph. Each connection can contain information such as distance, estimated travel time, transportation type, and cost. Graph-based algorithms are then used to identify an appropriate route according to the selected optimization criteria.

The project applies computational thinking through problem decomposition, abstraction, pattern recognition, and algorithm design. Python provides the implementation environment, while appropriate data structures such as dictionaries, lists, sets, queues, and priority queues support efficient processing. Object-oriented programming helps organize locations, routes, journeys, and bookings into reusable components. Functional programming techniques can be used for route filtering and cost calculations.

Where applicable, concurrency techniques can improve responsiveness during independent operations such as retrieving route information or processing external booking requests. Software quality is supported through unit testing, integration testing, type checking, code formatting, and automated validation. AI-assisted programming tools may also support specification preparation, code generation, debugging, and code review, with all generated code verified through testing.

The main objective of this project is to demonstrate how computational thinking and Python programming can solve a practical travel-planning problem through an organized, efficient, and maintainable software solution. The platform can be extended with live traffic information, multiple transportation modes, personalized recommendations, and real-time booking integrations.

Keywords: Computational Thinking, Python, Shortest Path, Graph Algorithms, Data Structures, Object-Oriented Programming, Travel Booking, Software Testing.

INDEX

S. No	TITLE	PAGE NO
1	Introduction	9
2	Problem Statement	10-11
3	System Architecture and Technologies	12-13
4	Problem Solving & Computational Thinking	14-17
5	Programming & Data Structures	18-19
6	Software Design & Engineering	20-21
7	Functional Programming & Concurrency	22
8	Software Quality & DevOps Practices	23-24
9	AI-Assisted Software Engineering	25-26
10	Algorithms, Complexity and Performance	27-28
11	Implementation and User Workflow	29
12	Testing, Results and Limitations	30-32
13	Conclusion and Future Enhancements	33
14	References	34

1. INTRODUCTION

Travel planning is an important real-world problem because users need to choose suitable routes while considering distance, travel time, cost, and transportation availability. When multiple routes connect a source and destination, manually comparing every possible route becomes difficult, particularly in a large transportation network.

The Smart Travelling Shortest Path Booking Platform addresses this problem by applying computational thinking and algorithmic techniques to travel planning. It models the transportation network as a graph, processes user input, identifies suitable paths, displays route information, and supports the booking process.

The platform combines problem-solving techniques with Python programming and software engineering principles. It demonstrates how a complex real-world problem can be divided into smaller tasks, represented using a simplified computational model, solved using suitable algorithms, and validated through systematic testing.

The project also demonstrates the importance of selecting suitable data structures, designing modular software, analyzing algorithm complexity, and using AI-assisted development responsibly.

Objectives

- Identify a suitable route between a given source and destination.
- Compare available routes using distance, estimated time, or cost.
- Apply graph algorithms to solve shortest-path problems.
- Organize route and booking information using suitable data structures.
- Develop modular and reusable Python components.
- Apply object-oriented and functional programming principles.
- Evaluate algorithm performance using time and space complexity.
- Validate the application using automated testing and software quality tools.
- Provide a convenient interface for route selection and booking.
- Demonstrate responsible AI-assisted software development.

2. PROBLEM STATEMENT

Travelers often face difficulties selecting an efficient route when several transportation connections are available between two locations. Manual route comparison can be time-consuming and may not identify the most suitable path according to distance, duration, or travel cost.

The problem becomes more challenging when the transportation network contains many locations and connections. A system must process the source and destination, represent the available connections, calculate possible routes, compare their costs, and present an appropriate result.

Therefore, the problem is to design and implement a Smart Travelling Shortest Path Booking Platform that accepts travel details, models the transportation network, applies a suitable shortest-path algorithm, presents an efficient route, and supports booking-related operations.

Input

- Source location
- Destination location
- Available route connections
- Distance, estimated duration, or cost associated with each connection
- User-selected travel preferences
- Booking details, where applicable

Processing

- Validate the source and destination.
- Represent locations and connections using a graph.
- Identify possible paths through the network.
- Calculate the route weight according to the selected criterion.
- Find the minimum-weight route.
- Display route details and process booking information.

Output

- Recommended route
- Sequence of locations
- Total distance or estimated journey time
- Estimated cost, if supported
- Alternative routes, if implemented
- Booking confirmation or booking status, if implemented

The system aims to improve route selection by automating calculations and organizing travel information into a consistent workflow.

3.SYSTEM ARCHITECTURE AND TECHNOLOGIES

The application follows a modular architecture in which the user interface, route-processing logic, and booking operations are separated.

Main modules

1. **User Interface Module:** Accepts the source, destination, travel preferences, and booking details.
2. **Input Validation Module:** Checks whether the locations and booking details are valid.
3. **Route Management Module:** Stores and retrieves transportation connections.
4. **Shortest-Path Module:** Calculates the best route using a suitable graph algorithm.
5. **Booking Module:** Manages booking requests, journey details, and booking status.
6. **Data Management Module:** Stores or retrieves route and booking information.
7. **Testing and Validation Module:** Checks the correctness and reliability of the application.

Technologies

Technology	Purpose
Python	Main programming language and algorithm implementation
Graphs	Represent locations and transportation connections
Dictionaries and lists	Store route and journey information
Heap-based priority queue	Efficiently select the next lowest-cost location in Dijkstra's algorithm
Object-oriented programming	Organize route, location, and booking components
Pytest	Automate unit and integration tests
Type hints	Improve code clarity and enable static checking
Git and GitHub	Version control and collaborative development

Docker and CI/CD	Optional containerization and automated validation
AI coding assistant	Support planning, coding, debugging, and review

The actual technologies used in the submitted implementation should be listed in the final report. Optional technologies should not be presented as completed features unless they were implemented.

System workflow

User Input → Input Validation → Transportation Graph → Shortest-Path Algorithm → Route Comparison → Route Display → Booking Processing → Confirmation

4. PROBLEM SOLVING AND COMPUTATIONAL THINKING

Computational thinking is a problem-solving approach used to understand complex problems and develop efficient computational solutions. It involves identifying the problem, breaking it into smaller parts, removing unnecessary details, recognizing repeated patterns, and designing suitable algorithms.

In the Smart Travelling Shortest Path Booking Platform, computational thinking is used to simplify travel planning, represent transportation connections, identify suitable routes, and support the booking process. The project applies four major computational thinking concepts: problem decomposition, abstraction, pattern recognition, and algorithm design.

4.1 Problem Decomposition

Meaning: Problem decomposition is the process of dividing a large and complex problem into smaller, manageable subproblems.

Where it is used in this project:

The complete travel-planning problem is divided into the following tasks:

- Collect source and destination details.
- Validate the user input.
- Store locations and transportation connections.
- Calculate suitable routes.
- Compare route distances, times, or costs.
- Display the selected route.
- Process booking details.
- Test the individual modules and the complete workflow.

Each task can be developed and tested independently. For example, the shortest-path algorithm can be tested without requiring the booking interface.

Example: Instead of developing the entire travel application in one large program, separate Python modules can be created for route management, shortest-path calculation, booking management, and input validation.

Benefit: Decomposition reduces complexity, improves maintainability, simplifies debugging, and supports parallel development.

4.2 Abstraction

Meaning: Abstraction means representing only the important details of a real-world problem while ignoring unnecessary information.

Where it is used in this project:

A real transportation network contains many details, including roads, buildings, traffic signals, geographical coordinates, and vehicle movements. The project simplifies this information into a graph.

- A **node (vertex)** represents a location.
- An **edge** represents a direct connection between two locations.
- A **weight** represents distance, estimated travel time, or cost.

For example, the application may represent three locations as A, B, and C, with connections A–B and B–C. The algorithm works with these locations and edge weights rather than modeling every physical detail of the journey.

Benefit: Abstraction reduces the amount of information the program must process and allows graph algorithms to solve the route-selection problem efficiently.

4.3 Pattern Recognition

Meaning: Pattern recognition means identifying similarities, repeated structures, or recurring situations in a problem.

Where it is used in this project:

The transportation network contains repeated patterns:

- Many locations connect to multiple neighboring locations.
- Each connection can be represented using the same structure: starting location, destination location, and weight.
- Different journeys follow the same general process of route calculation.
- Multiple route choices can be compared using the same cost-minimization rule.

For example, if A connects to B and C, and both B and C connect to D, the system recognizes two possible paths from A to D:

- A → B → D
- A → C → D

The algorithm compares their total weights using the same rule rather than requiring separate logic for each route.

Pattern recognition also helps identify recurring booking-validation rules, repeated route calculations, and common error conditions.

Benefit: It enables reusable code, consistent processing, and the application of established graph algorithms to different journeys.

4.4 Algorithm Design and Selection

Meaning: Algorithm design is the process of creating a clear sequence of steps to solve a computational problem. Algorithm selection involves choosing a suitable method based on the problem requirements and input characteristics.

Where it is used in this project:

The system needs an algorithm to identify a minimum-weight route between two locations.

Possible algorithms include:

- **Breadth-First Search (BFS):** Suitable for finding a path with the fewest edges in an unweighted graph.
- **Dijkstra's algorithm:** Suitable for finding a minimum-distance, minimum-time, or minimum-cost path when all edge weights are non-negative.
- *A search:** Can reduce the search area when a suitable heuristic is available.
- **Dynamic programming:** Can be useful for related optimization problems with overlapping subproblems, but it is not automatically required for a standard shortest-path implementation.

For a transportation graph with non-negative edge weights, Dijkstra's algorithm is an appropriate baseline.

Algorithm steps:

1. Assign distance zero to the source and infinity to all other locations.

2. Insert the source into a priority queue.
3. Remove the location with the smallest known distance.
4. Examine its neighboring locations.
5. Calculate the candidate distance to each neighbor.
6. If a shorter distance is found, update the distance and predecessor.
7. Repeat until the destination is reached or no reachable locations remain.
8. Reconstruct the route using the predecessor information.

Benefit: The algorithm automates route comparison and identifies an optimal path under the selected weight criterion.

4.5 Other Algorithmic Patterns

The project can demonstrate the following patterns where appropriate:

- **Greedy:** Dijkstra's algorithm repeatedly selects the unsettled location with the smallest tentative distance.
- **Divide-and-conquer:** May be used for independent data-processing tasks, but it is not necessary for the basic shortest-path solution.
- **Dynamic programming:** May be applied to related problems that contain overlapping subproblems.
- **Backtracking:** Can enumerate alternative routes in a small network, subject to constraints.
- **Branch-and-bound:** Can prune partial routes whose accumulated cost already exceeds a known solution, when the pruning conditions are valid.

These techniques should be described as implemented only if the source code actually uses them.

5. PROGRAMMING AND DATA STRUCTURES

Python is used to implement the route calculation and booking workflow. Selecting appropriate data structures improves readability, performance, and memory use.

5.1 Python Programming

Python supports modular development through functions, classes, modules, exceptions, comprehensions, and built-in collections.

Example graph representation:

```
graph = {
    "A": [("B", 5), ("C", 2)],
    "B": [("A", 5), ("D", 4)],
    "C": [("A", 2), ("D", 3)],
    "D": [("B", 4), ("C", 3)]
}
```

Each dictionary key represents a location. Its value is a list of neighboring locations and their corresponding edge weights.

For example, the connection from A to B has weight 5, while the connection from A to C has weight 2.

5.2 Data Structures Used

Data structure	Application
Dictionary	Store adjacency lists, distances, and predecessors
List	Store neighboring locations and reconstructed paths
Set	Track visited or settled locations
Tuple	Represent a neighbor and its edge weight

Heap-based priority queue	Retrieve the next location with the smallest tentative distance
Class objects	Represent locations, routes, journeys, and bookings
Database tables, if used	Persist route and booking records

5.3 Pythonic Programming Techniques

The implementation can use:

- Comprehensions to filter or transform route data.
- Iterators to process route records one at a time.
- Generators to yield alternative routes without storing every result at once.
- Decorators for reusable logging, timing, or validation behavior when appropriate.
- Context managers to manage files, database sessions, or other resources safely.
- Exception handling to manage invalid inputs, missing routes, and booking errors.
- Type hints to clarify function inputs and outputs.

These techniques should be included in the practical demonstration when they are present in the actual implementation.

5.4 Performance-Aware Programming

An adjacency-list graph avoids storing every possible connection in a dense matrix when the transportation network is sparse. A priority queue can improve the efficiency of shortest-path processing.

The application should also avoid recalculating an unchanged route unnecessarily, loading more data than required, or repeatedly performing expensive database queries.

6. SOFTWARE DESIGN AND ENGINEERING

Software design organizes the application into understandable, reusable, and independently testable components.

6.1 Object-Oriented Programming

Object-oriented programming (OOP) organizes software around classes and objects.

Possible classes include:

- Location: Stores a location identifier and related details.
- Route: Stores connected locations, distance, time, and cost.
- RoutePlanner: Performs shortest-path calculations.
- Booking: Stores journey and booking information.
- BookingService: Validates and processes booking operations.

A RoutePlanner object can use route information supplied by a graph or route repository. A BookingService can use the selected route when creating a booking.

6.2 OOP Principles

Encapsulation: Route and booking data are managed through defined class interfaces rather than being modified unpredictably throughout the program.

Abstraction: A caller can request a shortest route without needing to understand every internal step of the graph algorithm.

Inheritance: A shared base class may be useful if the application contains several related transportation types with genuinely common behavior. Inheritance is not required simply to demonstrate OOP.

Composition: A booking service can contain or use a route planner and a booking repository. This is often more flexible than a deep inheritance hierarchy.

6.3 Type Hints and Generic Programming

Type hints clarify the expected inputs and outputs of functions.

For example:

```
def calculate_total(distance: float, rate: float) -> float:
    return distance * rate
```

This function accepts distance and rate values and returns a calculated total.

Generic types can be useful when implementing reusable data structures or utilities that operate on different data types.

6.4 Design Patterns

Possible patterns include:

- Strategy pattern: Select between different route-planning algorithms or optimization criteria.
- Repository pattern: Separate booking and route data access from business logic.
- Factory pattern: Create suitable transport or booking objects when the application has multiple object types.

The design should use patterns only when they solve an actual problem in the code.

6.5 Modular Design

The program can be organized into modules such as:

```
smart_travel/
    main.py
    models.py
    route_planner.py
    booking_service.py
    validation.py
    tests/
        test_routes.py
        test_bookings.py
```

This structure is an example and should be adjusted to match the actual project files.

7. FUNCTIONAL PROGRAMMING AND CONCURRENCY

7.1 Functional Programming

Functional programming emphasizes functions, predictable behavior, and reduced dependence on mutable shared state.

For example:

```
def calculate_cost(distance: float, rate: float) -> float:

    return distance * rate
```

This function returns a result based on its inputs without modifying external state. It is a pure function when the calculation has no hidden side effects.

Other applicable techniques include:

- Higher-order functions for reusable filtering and transformation.
- map() and filter() for processing collections where they improve clarity.
- List comprehensions for readable data transformation.
- Immutable tuples for representing fixed route records.
- Small, independent functions for route scoring and booking validation.

7.2 Concurrency

Concurrency allows multiple tasks to make progress during overlapping periods. Its suitability depends on the workload.Threading: Useful for I/O-bound tasks, such as waiting for external route information or booking-service responses.Multiprocessing: Useful for CPU-bound calculations that can be divided into independent processes. Process creation and communication introduce overhead, so multiprocessing is not automatically beneficial for a small route graph.Asynchronous programming: Useful when the application needs to handle many simultaneous network or database operations without blocking the event loop.

7.3 CPU-Bound, I/O-Bound Workloads and the GIL

A CPU-bound task spends most of its time performing calculations. An I/O-bound task spends much of its time waiting for network, database, or file operations.In conventional CPython builds, the Global Interpreter Lock (GIL) limits simultaneous execution of Python bytecode across threads. Threads can still be effective for I/O-bound operations.

8. SOFTWARE QUALITY AND DEVOPS PRACTICES

Software quality ensures that the route-planning and booking features work correctly, handle errors, and remain maintainable as the application grows.

8.1 Unit Testing

Unit tests verify individual functions and classes.

Examples:

- Check that the shortest route is returned for a known graph.
- Check that the source and destination are validated.
- Check that an unreachable destination produces an appropriate result.
- Check that travel cost is calculated correctly.
- Check that invalid booking details are rejected.

8.2 Integration Testing

Integration tests verify that modules work together.

For example, the test can submit a source and destination, obtain the calculated route, pass it to the booking service, and verify the resulting booking details.

8.3 End-to-End Testing

End-to-end tests validate the complete user journey, from entering travel information to viewing the route and receiving a booking status.

These tests are applicable when the project has a working user interface and booking workflow.

8.4 Pytest and Property-Based Testing

Pytest supports readable automated tests. Property-based testing tools such as Hypothesis can generate many input cases to check general correctness properties.

For a non-negative weighted graph, useful properties include:

- Every consecutive pair of locations in the returned path must be connected.

- The total path weight must equal the sum of its edge weights.
- If a route is returned, its cost must be no greater than the cost of any other valid route in a small exhaustively testable graph.
- A reachable source must have distance zero from itself.

8.5 Code Coverage, Type Checking, Linting and Formatting

- Coverage tools: Measure which parts of the program are exercised by tests.
- Mypy or Pyright: Check type consistency.
- Ruff: Detect common code-quality issues and support linting and formatting workflows.
- Git: Track changes and support safe collaboration.
- GitHub Actions: Can run automated tests and checks when code is pushed.

8.6 CI/CD and Containerization

If implemented, a continuous integration pipeline can install dependencies and run tests and static checks automatically. Docker can package the application and its runtime dependencies in a consistent environment.

These practices make it easier to reproduce the development environment and identify problems before deployment.

9. AI-ASSISTED SOFTWARE ENGINEERING

AI-assisted software engineering uses AI tools to support development while retaining human responsibility for correctness, security, and quality.

9.1 Effective Prompting

A useful prompt should state the problem, input format, expected output, constraints, algorithm requirements, and testing requirements.

Example:

"Implement a Python shortest-path function using Dijkstra's algorithm for a graph with non-negative weights. Return the minimum total cost and reconstructed path. Handle invalid source locations and unreachable destinations. Include type hints and pytest tests for normal and edge cases."

9.2 Context Management

Providing relevant source files, project structure, dependencies, error messages, and existing function signatures helps the AI tool produce code that fits the application.

Only the context necessary for the task should be supplied, and secrets or credentials should not be shared.

9.3 Specification-First Development

Before writing code, define:

- The problem and intended users.
- Valid inputs and expected outputs.
- Constraints and edge cases.
- Route-selection rules.
- Booking requirements.
- Acceptance tests and performance expectations.

The code is then developed against these requirements.

9.4 AI-Assisted Code Generation

AI tools can help generate initial functions, test cases, documentation, type hints, and explanations of errors. The generated output should be reviewed before integration.

9.5 Critical Review and Validation

AI-generated code may contain logical errors, incorrect assumptions, inefficient algorithms, or security weaknesses. Therefore, the developer must:

1. Review the generated code.
2. Verify the selected algorithm.
3. Run unit and integration tests.
4. Check complexity and edge cases.
5. Use static analysis and type checking where appropriate.
6. Compare outputs with known examples.
7. Document meaningful changes and limitations.

AI assistance supports development; it does not replace understanding or testing.

10. Algorithms, Complexity and Performance

Algorithm analysis estimates how execution time and memory requirements grow as the input becomes larger.

Let (V) be the number of locations (vertices) and (E) the number of transportation connections (edges).

10. ALGORITHMS, COMPLEXITY AND PERFORMANCE

10.1 Dijkstra's Algorithm

Using an adjacency list and a binary heap, a common implementation of Dijkstra's algorithm has time complexity:

[
O((V+E)\log V)
]

Its graph storage requires (O(V+E)) space. Additional distance, predecessor, and priority-queue storage is generally (O(V)) for a standard implementation, subject to implementation details.

10.2 Comparison of Algorithms

Algorithm	Suitable situation	Typical time complexity
BFS	Unweighted graph	(O(V+E))
Dijkstra with binary heap	Non-negative edge weights	(O((V+E)\log V))
Bellman–Ford	Negative edge weights, without reachable negative cycles	(O(VE))
Floyd–Warshall	Shortest paths between all pairs	(O(V^3))

Dijkstra's algorithm should not be used when negative edge weights invalidate its assumptions.

10.3 Space Complexity

An adjacency list requires (O(V+E)) space. Distance and predecessor dictionaries require (O(V)) additional space. A reconstructed route may require up to (O(V)) space.

10.4 Amortized and Expected Complexity

Amortized complexity describes the average cost per operation over a sequence of operations. For example, appending to a Python list is amortized (O(1)), although an individual append may require resizing.

Expected complexity describes performance averaged over a probability distribution or randomized behavior. It may be relevant when using randomized algorithms, hash-based operations under suitable assumptions, or randomized test inputs.

These concepts should be distinguished from worst-case algorithm complexity.

10.5 Performance Evaluation

The system can be evaluated using:

- Number of locations and connections.
- Route-calculation execution time.
- Memory consumption.
- Correctness of returned routes.
- Behavior for unreachable destinations.
- Response time under concurrent requests, if supported.

Performance results should be obtained by running the actual application on recorded test cases. Numerical performance claims should not be invented.

11. IMPLEMENTATION AND USER WORKFLOW

The proposed user workflow is:

1. Open the Smart Travelling Shortest Path Booking Platform.
2. Enter the source and destination.
3. Select the optimization criterion, such as distance or estimated time, if supported.
4. Validate the travel details.
5. Retrieve the transportation connections.
6. Execute the shortest-path algorithm.
7. Display the recommended route and its calculated details.
8. Review the route and any available alternatives.
9. Enter the required booking information.
10. Submit the booking request.
11. Display the booking status or confirmation.

The route-planning module should remain independent of the user interface so that the same algorithm can be tested separately and reused by other components.

A booking should only be described as confirmed when the application has successfully completed the relevant booking operation. If the project uses demonstration data rather than a live transport provider, that limitation must be stated.

12. TESTING, RESULTS AND LIMITATIONS

12.1 Testing Scenarios

Test case	Expected result
Valid source and destination	A suitable route is returned
Source equals destination	Zero-cost route containing the source
Destination is unreachable	Appropriate no-route result
Invalid location	Clear validation error
Multiple possible paths	Minimum-weight path is selected
Different edge weights	Route follows the selected optimization criterion
Invalid booking details	Booking request is rejected
Valid booking details	Booking status is returned correctly
Empty graph	Appropriate error or no-route result

12.2 Results

The project should demonstrate that the selected algorithm produces the expected shortest path for known test graphs and that route and booking components behave correctly for valid and invalid inputs.

The final report should include actual test counts, coverage percentage, execution times, screenshots, and sample route outputs where these have been measured. The results should reflect the implementation rather than assumed values.

12.3 Limitations

Potential limitations include:

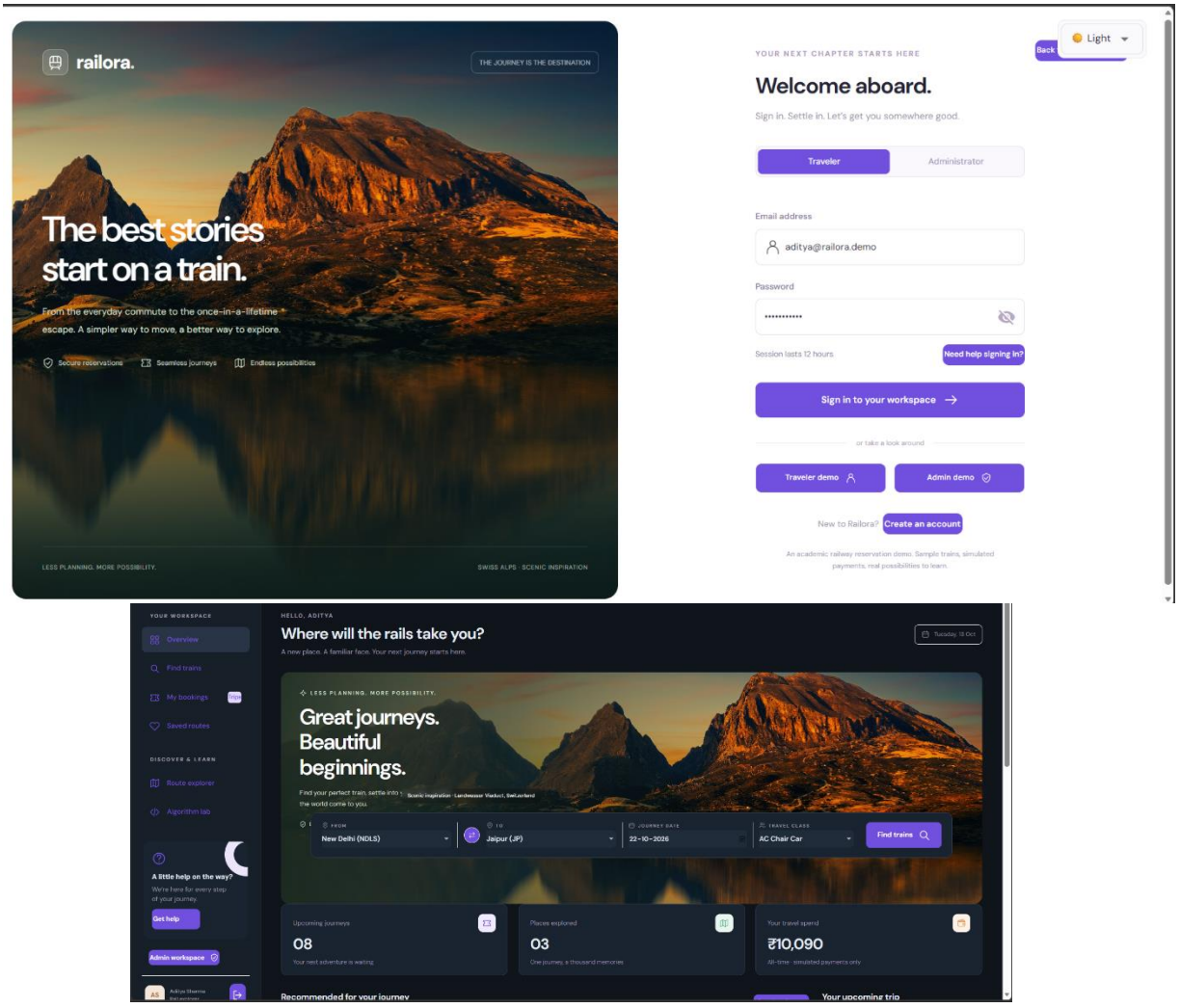
- Static route data may not reflect live road conditions.
- Estimated journey times may differ from actual travel times.

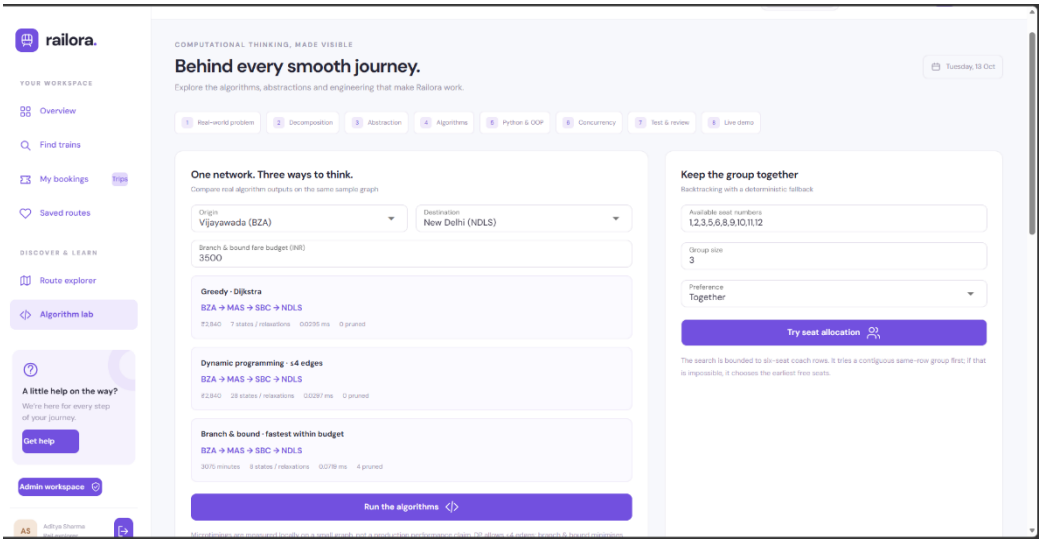
- The recommended route is only as reliable as the supplied network and edge weights.
- Booking functionality may be limited if external transport-provider APIs are unavailable.
- Large networks may require caching, geographic indexing, or specialized routing algorithms.
- Concurrent processing may introduce synchronization and error-handling complexity.
- AI-generated code may require additional review and testing.

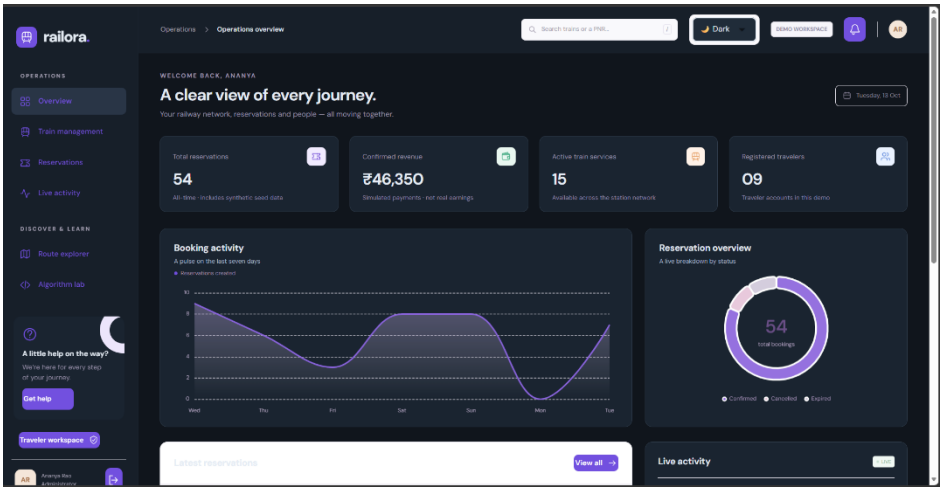
12.4 Future Enhancements

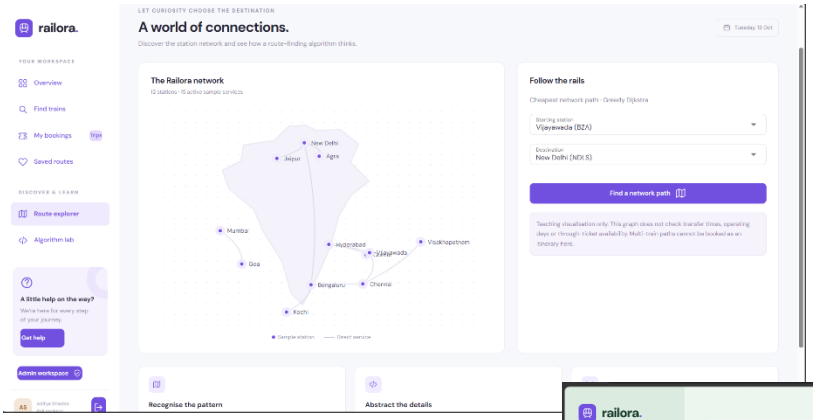
Future development may include real-time traffic updates, GPS integration, public transportation schedules, multiple transport modes, personalized route recommendations, secure payment integration, booking cancellation, route caching, mobile support, and live transport-provider integration.

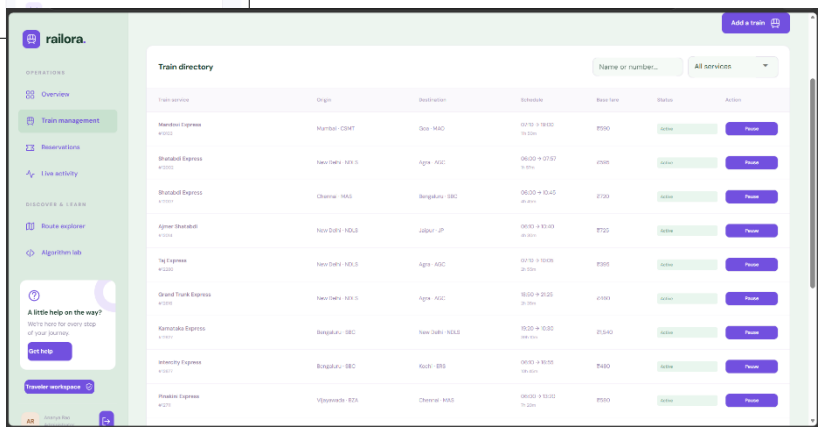
12.5 IMAGES




railora.
COMPUTATIONAL THINKING, MADE VISIBLE
Behind every smooth journey.
Explore the algorithms, abstractions and engineering that make Railora work.
Tuesday, 13 Oct
YOUR WORKSPACE
Overview
Find trains
My bookings
Saved routes
DISCOVER & LEARN
Route explorer
Algorithm lab
A little help on the way?
We're here for every step of your journey.
Get help
Admin workspace
1 Real-world problem
2 Decomposition
3 Abstraction
4 Algorithms
5 Python & OOP
6 Concurrency
7 Test & review
8 Live demo
One network. Three ways to think.
Compare real algorithm outputs on the same sample graph
Origin
Vijayawada (BZA)
Destination
New Delhi (NDLS)
Branch & bound fare budget (INR)
3500
Greedy - Dijkstra
BZA → MAS → SBC → NDLS
₹2,840 7 states / relaxations 0.0295 ms 0 pruned
Dynamic programming - s4 edges
BZA → MAS → SBC → NDLS
₹2,840 28 states / relaxations 0.0297 ms 0 pruned
Branch & bound - fastest within budget
BZA → MAS → SBC → NDLS
3070 minutes 8 states / relaxations 0.0718 ms 4 pruned
Run the algorithms
Keep the group together
Backtracking with a deterministic fallback
Available seat numbers
1,2,3,5,6,8,9,10,11,12
Group size
3
Preference
Together
Try seat allocation
The search is bounded to six-seat coach rows. It tries a contiguous same-row group first; if that is impossible, it chooses the earliest free seats.


railora.
Operations
Operations overview
Dark
DEMO WORKSPACE
OPERATIONS
Overview
Train management
Reservations
Live activity
DISCOVER & LEARN
Route explorer
Algorithm lab
A little help on the way?
We're here for every step of your journey.
Get help
Traveler workspace
WELCOME BACK, ANANYA
A clear view of every journey.
Your railway network, reservations and people — all moving together.
Tuesday, 13 Oct
Total reservations
54
All-time · includes synthetic seed data
Confirmed revenue
₹46,350
Simulated payments · not real earnings
Active train services
15
Available across the station network
Registered travelers
09
Traveler accounts in this demo
Booking activity
A pulse on the last seven days
Reservations created
10
8
6
4
2
0
Wed
Thu
Fri
Sat
Sun
Mon
Tue
Reservation overview
A live breakdown by status
54
total bookings
Confirmed
Cancelled
Expired
Latest reservations
View all
Live activity
Live


railora.
LET CURIOSITY CHOOSE THE DESTINATION
A world of connections.
Discover the station network and see how a route-finding algorithm thinks.
Tuesday, 13 Oct
YOUR WORKSPACE
Overview
Find trains
My bookings
Trips
Saved routes
DISCOVER & LEARN
Route explorer
Algorithm lab
A little help on the way?
We're here for every step of your journey.
Get help
Admin workspace
The Railora network
12 stations · 15 active sample services
New Delhi
Agra
Jaipur
Mumbai
Goa
Hyderabad
Vijayawada
Visakhapatnam
Bengaluru
Chennai
Kochi
Sample station
Direct service
Follow the rails
Cheapest network path · Greedy Dijkstra
Starting station
Vijayawada (BZA)
Destination
New Delhi (NDLS)
Find a network path
Teaching visualisation only. This graph does not check transfer times, operating days or through-ticket availability. Multi-train paths cannot be booked as an itinerary here.
Recognise the pattern
Abstract the details


railora.
Add a train
Train directory
Name or number...
All services
OPERATIONS
Overview
Train management
Reservations
Live activity
DISCOVER & LEARN
Route explorer
Algorithm lab
A little help on the way?
We're here for every step of your journey.
Get help
Traveler workspace
Mandovi Express
Mumbai - CSMT
Goa - MAO
Pause
Bhatiabali Express
New Delhi - NDLS
Agra - AGC
Pause
Bhatiabali Express
Chennai - MAS
Bengaluru - SBC
Pause
Ajmer Shatabdi
New Delhi - NDLS
Jaipur - JP
Pause
Taj Express
New Delhi - NDLS
Agra - AGC
Pause
Grand Trunk Express
New Delhi - NDLS
Agra - AGC
Pause
Karnataka Express
Bengaluru - SBC
New Delhi - NDLS
Pause
Intercity Express
Bengaluru - SBC
Kochi - ERS
Pause
Pinakini Express
Vijayawada - BZA
Chennai - MAS
Pause

13. CONCLUSION AND FUTURE ENHANCEMENTS

The Smart Travelling Shortest Path Booking Platform demonstrates how computational thinking and Python programming can be applied to a practical travel-planning problem. The transportation network is modeled as a graph, and a suitable shortest-path algorithm is used to identify an efficient route according to the selected optimization criterion.

Problem decomposition helps divide the application into manageable modules, abstraction simplifies the transportation network, pattern recognition identifies reusable structures, and algorithm design provides a systematic approach to route optimization. Suitable data structures and object-oriented design support efficient processing and maintainable code, while functional programming techniques improve the organization of independent operations.

Testing, static analysis, version control, and optional CI/CD practices contribute to software reliability. AI-assisted development can support planning and implementation when its output is reviewed and validated carefully.

The project provides a foundation for more advanced travel-planning systems. Future enhancements can integrate live transportation data, multiple optimization objectives, external booking services, and personalized recommendations.

Overall, the project demonstrates the connection between computational thinking, algorithmic problem-solving, Python programming, software engineering, and practical application development.

14. REFERENCES

1. Python Software Foundation. *Python Documentation*. https://docs.python.org/3/

2. Python Software Foundation. *Python Data Structures*. https://docs.python.org/3/tutorial/datastructures.html

3. Python Software Foundation. *Python heapq — Heap Queue Algorithm*. https://docs.python.org/3/library/heapq.html

4. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. *Introduction to Algorithms*. MIT Press.

5. Pytest Documentation. https://docs.pytest.org/

6. Hypothesis Documentation. https://hypothesis.readthedocs.io/

7. Mypy Documentation. https://mypy.readthedocs.io/

8. Ruff Documentation. https://docs.astral.sh/ruff/

9. Git Documentation. https://git-scm.com/doc

10. Docker Documentation. https://docs.docker.com/

11. GitHub Actions Documentation. https://docs.github.com/actions